

# 從零開始到實際工作環境的 Cloud Run 終極指南

來源：<https://codelabs.developers.google.com/next26/ultimate-cloud-run-guide?hl=zh-tw>

步驟數：10

Cloud Nex 講座的程式碼實驗室：從零開始到正式環境的 Cloud Run 終極指南

---

# 目錄

1. [1. 簡介](#)
2. [2. 從映像檔部署](#)
3. [3. 從來源部署](#)
4. [4. 回溯和預覽連結](#)
5. [5. Developer Knowledge 與 MCP 伺服器](#)
6. [6. 使用 Secret Manager](#)
7. [7. 連線至虛擬私有雲](#)
8. [8. 部署 ADK 代理](#)
9. [9. 清除所用資源](#)
10. [10. 恭喜！](#)

步驟 1 · 約 0 分鐘

## 1. 簡介

### 從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

info

### 從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

啟用

這個程式碼研究室會帶您瞭解如何開始使用 Cloud Run 的基本知識。您將瞭解如何使用其他功能，包括虛擬私有雲存取權、Secret Manager 和 ADK，適用於 Cloud Run 上代管的 AI 代理程式。

## 課程內容

- 部署 nginx 映像檔
- 從原始碼部署
- 復原部署項目
- 預覽部署作業
- 使用 Developer Knowledge MCP 伺服器工具
- 搭配使用 Secret Manager 與 Cloud Run
- 連線至虛擬私有雲中的內部 Cloud Run 服務
- 將 ADK 代理部署至 Cloud Run

## 軟硬體需求

- 網路瀏覽器，例如 [Chrome](#)
- 已啟用計費功能的 Google Cloud 雲端專案

## 建立 Google Cloud 專案

1. 在 [Google Cloud 控制台](#) 的專案選取器頁面中，[選取或建立 Google Cloud 專案](#)。
2. 確認 Cloud 專案已啟用計費功能。瞭解如何[檢查專案是否已啟用計費功能](#)。

## 啟動 Cloud Shell

Cloud Shell 是在 Google Cloud 中運作的指令列環境，已預先載入必要工具。

1. 按一下 Google Cloud 控制台頂端的「啟用 Cloud Shell」。
2. 連至 Cloud Shell 後，請驗證您的驗證：

```
gcloud auth list
```

3. 確認專案已設定完成：

```
gcloud config get project
```

4. 如果專案未如預期設定，請設定專案：

```
export PROJECT_ID=<YOUR_PROJECT_ID>  
gcloud config set project $PROJECT_ID
```

## 設定環境變數

本程式碼研究室使用下列環境變數。

首先，請設定所在區域。

```
export REGION=<YOUR_REGION>
```

接著，確認 PROJECT\_ID 和 REGION

```
echo "PROJECT_ID: $PROJECT_ID | REGION: $REGION"
```

---

步驟 2 · 約 0 分鐘

## 2. 從映像檔部署

在本節中，您將直接從 Docker Hub 部署標準 nginx 映像檔。您將設定公開存取權，並將容器連接埠設為 80。

### 1. 部署 nginx 服務：

```
gcloud run deploy nginx-service \  
  --image=nginx \  
  --allow-unauthenticated \  
  --port=80 \  
  --region=$REGION
```

### 2. 部署完成後，指令輸出內容會提供服務網址。在瀏覽器中開啟該網址，即可看到「Welcome to nginx!」頁面。

---

步驟 3 · 約 0 分鐘

### 3. 從來源部署

```
mkdir color-app && cd $_
```

建立名為 `requirements.txt` 的檔案，並加入以下內容：

```
Flask>=2.0.0  
gunicorn>=20.0.0
```

建立名為 `main.py` 的檔案，並加入以下內容：

```

import os
from flask import Flask, render_template_string

app = Flask(__name__)

TEMPLATE = """
<!doctype html>
<html lang="en">
<head>
    <title>Cloud Run Traffic Revisions</title>
    <style>
        body {
            display: flex;
            justify-content: center;
            align-items: center;
            min-height: 50vh;
            background-color: darkseagreen;
            font-family: sans-serif;
        }
        .content {
            background-color: rgba(255, 255, 255, 0.8); /* Semi-transparent white background
*/
            padding: 2em;
            border-radius: 8px;
            text-align: center;
            box-shadow: 0 4px 8px rgba(0,0,0,0.1);
        }
    </style>
</head>
<body>
    <div class="content">
        <!-- ROLLBACK DEMO: change this text to "gray" -->
        <p>background color: <strong>darkseagreen</strong></p>
    </div>
</body>
</html>
"""

@app.route('/')
def main():

    return render_template_string(TEMPLATE)

if __name__ == '__main__':
    port = int(os.environ.get('PORT', 8080))
    app.run(debug=True, host='0.0.0.0', port=port)

```

現在請執行下列指令。

```
gcloud run deploy \  
  --allow-unauthenticated \  
  --region $REGION
```

---

## 4. 回溯和預覽連結

**附註：**每次變更時，Cloud Run 都會建立新的修訂版本。修訂版本無法變更。

在本節中，您會導入錯誤，並瞭解如何在調查修正方式時，復原至先前的修訂版本。

1. 首先，請記錄目前處理流量的修訂版本名稱，因為該版本不含錯誤。

```
GOOD_REVISION=$(gcloud run revisions list --service color-app \
  --region $REGION --format 'value(REVISION)')
```

2. 在 color-app `main.py` 檔案中，搜尋「ROLLBACK DEMO」，然後將該行更新為下列內容：

```
<p>background color: <strong>gray</strong></p>
```

3. 現在請重新執行 `gcloud run deploy`。請注意先前設定的使用方式。

現在您已部署錯誤，可以返回來源、進行變更或執行 git 還原，然後建構、觸發新版本等。不過，您可能會在過程中引入錯誤。

較安全的方式是復原。

4. 如要復原至先前修訂版本，請執行下列指令：

```
gcloud run services update-traffic color-app \
  --to-revisions=$GOOD_REVISION=100 \
  --region=$REGION
```

現在您可以部署不會接收任何流量的新修訂版本。

5. 現在請將文字改回 `darkseagreen`，修正錯誤。

```
<p>background color: <strong>darkseagreen</strong></p>
```

6. 並部署該版本來驗證修正內容。請注意，這個修訂版本不會收到任何流量，因為 100% 的流量都已固定分配給 `GOOD_REVISION`

```
gcloud run deploy color-app --no-traffic --tag bugfix --region $REGION
```

7. 驗證 Deployment

你會發現網址略有不同。造訪該網址時，您會看到這個部署作業中的錯誤修正。

8. 將流量傳送回最新修訂版本。

現在請將流量重新設回最新修訂版本。

```
gcloud run services update-traffic color-app \  
  --to-latest \  
  --region=$REGION
```

並刪除修訂版本標記

```
gcloud run services update-traffic color-app \  
  --remove-tags=bugfix \  
  --region=$REGION
```

詳情請參閱[說明文件](#)。

---

步驟 5 · 約 0 分鐘

## 5. Developer Knowledge 與 MCP 伺服器

[開發人員知識 MCP 伺服器](#)可讓 AI 輔助開發工具搜尋 Google 官方開發人員說明文件，並擷取 Firebase、Google Cloud、Android、地圖等 Google 產品的資訊。將 AI 應用程式直接連結至官方文件庫，確保您取得的程式碼和指引都是最新資訊，並以權威的脈絡為依據。

請按照[安裝指南](#)操作，授予 AI 代理存取 Developer Knowledge MCP 伺服器的權限。

安裝完成後，您可以向 AI 代理詢問說明文件中最新功能的問題，這些功能可能是在模型訓練日期截止後才推出。

舉例來說，如果您查看 [Cloud Run 版本資訊](#)，會看到 [2026 年 2 月 24 日](#) 的項目，內容是「使用 Cloud Run 服務健康狀態 (搶先版)，部署高可用性的多區域 Cloud Run 服務，並針對外部流量自動容錯移轉和容錯回復」。

現在可以向 AI 代理詢問「請進一步說明這項新的 Cloud Run 功能，瞭解如何進行多區域自動容錯移轉」。

---

## 6. 使用 Secret Manager

在 Cloud Run 中公開密鑰的方式有 3 種：

1. 以環境變數形式 (鎖定在部署時提取的版本)。
2. 掛接為檔案磁碟區 (持續更新至最新版本)。
3. 在程式碼中使用 Secret Manager 用戶端程式庫。

在本節中，您將使用專屬服務帳戶，將密鑰公開為環境變數。

1. 建立名為「my-secret」的新密鑰：

```
gcloud secrets create my-secret --replication-policy="automatic"
```

2. 將密鑰值新增為新版本：

```
echo -n "my precious" | gcloud secrets versions add my-secret --data-file=-
```

3. 為顏色應用程式建立專用服務帳戶：

```
gcloud iam service-accounts create color-app-sa \  
  --display-name="Color App Service Account"
```

4. 授予專屬服務帳戶密鑰存取權。

**注意：**身分需要「Secret 存取者」角色 (roles/secretmanager.secretAccessor)，而非「檢視者」角色！檢視者角色只允許使用者在 Google Cloud 中查看密碼，但無法查看內容。

```
gcloud secrets add-iam-policy-binding my-secret \  
  --member="serviceAccount:color-app-sa@${PROJECT_ID}.iam.gserviceaccount.com" \  
  --role="roles/secretmanager.secretAccessor"
```

5. 再次部署。現在服務可以存取 MY\_SECRET 環境變數：

```
gcloud run deploy color-app \  
  --source . \  
  --update-secrets=MY_SECRET=my-secret:latest \  
  --service-account=color-app-sa@${PROJECT_ID}.iam.gserviceaccount.com \  
  --region=$REGION
```

## 7. 連線至虛擬私有雲

在本節中，您將設定下列架構：

- 無法從公開網際網路存取的私人後端
- 透過直接虛擬私有雲輸出與後端通訊的公開前端

本範例會使用預設網路和子網路。

**必要條件：**請確認子網路上已啟用 Private Google Access，這樣虛擬私有雲才能將內部要求轉送至 Cloud Run 服務。

```
gcloud compute networks subnets update default \
  --region=$REGION \
  --enable-private-ip-google-access
```

### 1. 為這個部分建立資料夾

```
mkdir ../vpc-demo
cd ../vpc-demo
```

### 2. 建立私有後端服務

```
mkdir backend
touch backend/app.js
touch backend/package.json
```

在 `backend/app.js` 檔案中，新增下列內容：

```
const http = require('http');

const server = http.createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'text/plain' });
  res.end('Hello World from the Private Backend!');
});

const port = process.env.PORT || 8080;
server.listen(port, () => {
  console.log(`Private backend listening on port ${port}`);
});
```

在 `backend/package.json` 檔案中，新增下列內容：

```
{
  "name": "backend",
  "scripts": {
    "start": "node app.js"
  }
}
```

3. 部署僅限內部存取的私人後端：

```
gcloud run deploy private-backend \  
  --source ./backend \  
  --region $REGION \  
  --ingress internal \  
  --no-allow-unauthenticated
```

4. 請記下後端網址。稍後您會將這個網址提供給前端應用程式。

```
export PROJECT_NUMBER=$(gcloud projects describe $PROJECT_ID --format='value(projectNumber)')  
  
export BACKEND_URL="https://private-backend-${PROJECT_NUMBER}.${REGION}.run.app"
```

5. 建立前端應用程式

```
mkdir frontend  
touch frontend/app.js  
touch frontend/package.json
```

在 `frontend/app.js` 檔案中，新增下列內容：

```

const http = require('http');

const server = http.createServer(async (req, res) => {
  const backendUrl = process.env.BACKEND_URL;

  if (!backendUrl) {
    res.writeHead(500, { 'Content-Type': 'text/plain' });
    return res.end('Error: BACKEND_URL environment variable is missing.');
```

```

  }

  try {
    // Fetch the OIDC token from the Metadata server
    const tokenResponse = await
fetch(`http://metadata.google.internal/computeMetadata/v1/instance/service-
accounts/default/identity?audience=${backendUrl}`, {
    headers: { 'Metadata-Flavor': 'Google' }
  });

    if (!tokenResponse.ok) {
      throw new Error(`Failed to fetch identity token: ${tokenResponse.statusText}`);
    }
    const token = await tokenResponse.text();

    // Ping the backend with the token
    const response = await fetch(backendUrl, {
      headers: { 'Authorization': `Bearer ${token}` }
    });
    const text = await response.text();

    res.writeHead(200, { 'Content-Type': 'text/plain' });
    res.end(`Frontend successfully routed through VPC. Backend says: "${text}"`);
  } catch (error) {
    res.writeHead(500, { 'Content-Type': 'text/plain' });
    res.end(`Frontend failed to reach the backend. Error: ${error.message}`);
  }
});
```

```

const port = process.env.PORT || 8080;
server.listen(port, () => {
  console.log(`Public frontend listening on port ${port}`);
});
```

6. 在 `frontend/package.json` 檔案中，新增下列內容：

```

{
  "name": "frontend",
  "scripts": {
    "start": "node app.js"
  }
}
```

## 7. 為前端服務建立專用服務帳戶：

```
gcloud iam service-accounts create frontend-sa \  
  --display-name="Frontend Service Account"
```

## 8. 授予 Cloud Run Invoker 角色

```
PROJECT_ID=$(gcloud config get project)  
  
gcloud projects add-iam-policy-binding $PROJECT_ID \  
  --member="serviceAccount:frontend-sa@${PROJECT_ID}.iam.gserviceaccount.com" \  
  --role="roles/run.invoker"
```

## 9. 現在使用直接虛擬私有雲輸出流量部署公開前端。我們將「`--vpc-egress=all-traffic`」設為強制將輸出要求傳送至虛擬私有雲：

```
gcloud run deploy public-frontend \  
  --source ./frontend \  
  --region $REGION \  
  --allow-unauthenticated \  
  --network default \  
  --subnet default \  
  --vpc-egress all-traffic \  
  --service-account=frontend-sa@${PROJECT_ID}.iam.gserviceaccount.com \  
  --set-env-vars BACKEND_URL=$BACKEND_URL
```

## 10. 驗證服務

- **測試前端：**使用 curl 公開前端網址。這時應該就能順利與後端通訊並傳回回應。

```
FRONTEND_URL=$(gcloud run services describe public-frontend --region $REGION --  
format='value(status.url)')  
curl $FRONTEND_URL
```

- **測試後端 (直接)：**嘗試從本機 (公開網際網路) 直接捲曲後端網址。由於 Ingress 僅限「內部」使用，且需要驗證，因此應會失敗並顯示 404 錯誤。

```
curl $BACKEND_URL
```

---

步驟 8 · 約 0 分鐘

## 8. 部署 ADK 代理

在本節中，您將瞭解 Python 建構套件如何支援 Agent Development Kit (ADK) 的預設進入點偵測。

您將建立下列資料夾結構：

```
adk-demo
├── my_agent
│   ├── __init.py__
│   └── agent.py
└── requirements.txt
```

### 1. 建立資料夾結構

```
mkdir ../adk-demo
cd ../adk-demo
mkdir my_agent
touch my_agent/__init.py__
touch my_agent/agent.py
touch requirements.txt
```

### 2. 在 `my_agent/__init.py__` 檔案新增下列內容：

```
from . import agent
```

### 3. 在 `my_agent/agent.py` 檔案新增下列內容：

```
from google.adk import Agent

root_agent = Agent(
    name="demo_agent",
    model="gemini-3-flash-preview",
    instruction="You are a helpful assistant for a Cloud Run demo."
)
```

### 4. 在 `requirements.txt` 檔案新增下列內容：

```
google-adk
```

### 4. 為代理程式建立專用服務帳戶：

```
gcloud iam service-accounts create agent-sa \
    --display-name="Agent Service Account"
```

### 5. 將 Vertex AI 使用者角色授予服務帳戶：

```
PROJECT_ID=$(gcloud config get-value project)

gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member="serviceAccount:agent-sa@${PROJECT_ID}.iam.gserviceaccount.com" \
  --role="roles/aiplatform.user"
```

## 6. 部署 ADK 代理

**注意：**您必須從根 `adk-demo` 資料夾部署。

您必須在可存取 Gemini API 的區域部署。在本範例中，這個值為 `us-west1`。

```
gcloud run deploy my-adk-agent-demo \
  --source . \
  --region us-west1 \
  --allow-unauthenticated \
  --service-account=agent-sa@${PROJECT_ID}.iam.gserviceaccount.com \
  --set-env-
vars="GOOGLE_GENAI_USE_VERTEXAI=TRUE,GOOGLE_CLOUD_PROJECT=$PROJECT_ID,GOOGLE_CLOUD_LOCATION=g
lobal" \
  --labels dev-tutorial=codelab-ultimate-cr26
```

## 2. 使用 cURL 呼叫端點

您會發現代理程式可立即做為正式版 API 使用。

將 Cloud Run 服務網址記錄到環境變數中。

```
AGENT_URL=$(gcloud run services describe my-adk-agent-demo \
  --region us-west1 \
  --format 'value(status.url)')
```

與代理建立工作階段

```
curl -X POST $AGENT_URL/apps/my_agent/users/u_123/sessions/s_123 -H "Content-Type:
application/json" -d '{"key1": "value1", "key2": 42}'
```

詢問什麼是 Cloud Run，並篩選回覆內容，只顯示代理程式的說法

```
curl -X POST $AGENT_URL/run \
-H "Content-Type: application/json" \
-d "{
  \"appName\": \"my_agent\",
  \"userId\": \"u_123\",
  \"sessionId\": \"s_123\",
  \"newMessage\": {
    \"role\": \"user\",
    \"parts\": [{ \"text\": \"What is Cloud Run?\"
  }}
}" | python3 -c "import sys, json; print(json.load(sys.stdin)[-1]['content']['parts'][0]
['text'])"
```

您應該會看到類似下列內容：

```
Hello! I am **demo_agent**, and I'm here to help you with your Cloud Run demo.
**Cloud Run** is a fully managed compute platform by Google Cloud that allows you to
run **containerized applications** in a serverless environment...
```

**注意：**如果發生錯誤，請查看 Cloud Run 記錄，您可能必須前往這個網址啟用 Agent Platform API：  
<https://console.developers.google.com/apis/api/aipplatform.googleapis.com/overview>

步驟 9 · 約 0 分鐘

## 9. 清除所用資源

如要避免系統持續向您的 Google Cloud 帳戶收取費用，您可以刪除整個專案 (如下所示)，或刪除在本程式碼研究室中建立的個別資源。

刪除 nginx、color-app、private-backend、public-frontend 服務

```
gcloud run services delete nginx-service --region $REGION --quiet
gcloud run services delete color-app --region $REGION --quiet
gcloud run services delete private-backend --region $REGION --quiet
gcloud run services delete public-frontend --region $REGION --quiet
```

刪除 ADK 代理程式 (注意：本範例部署在 us-west1 中)

```
gcloud run services delete my-adk-agent-demo --region us-west1 --quiet
```

移除儲存在 Secret Manager 中的密鑰：

```
gcloud secrets delete my-secret --quiet
```

刪除 Color App 服務帳戶

```
gcloud iam service-accounts delete color-app-sa@${PROJECT_ID}.iam.gserviceaccount.com --quiet
```

刪除 ADK Agent 服務帳戶

```
gcloud iam service-accounts delete agent-sa@${PROJECT_ID}.iam.gserviceaccount.com --quiet
```

(選用) 刪除專案

如果您是特地為本程式碼研究室建立新專案，可以刪除整個專案，確保所有資源一次移除：

```
# run only if you want to delete the entire project
gcloud projects delete $PROJECT_ID
```

---

步驟 10 · 約 0 分鐘

## 10. 恭喜！

您已完成程式碼研究室。您已瞭解如何開始使用 Cloud Run 的基本知識。

### 您學到的內容

- 部署 nginx 映像檔
  - 從原始碼部署
  - 復原部署項目
  - 預覽部署作業
  - 使用 Developer Knowledge MCP 伺服器工具
  - 搭配使用 Secret Manager 與 Cloud Run
  - 連線至虛擬私有雲中的內部 Cloud Run 服務
  - 將 ADK 代理部署至 Cloud Run
-